D

DuesPay

On-Chain Dues Collection for Classes & Associations

WHITEPAPER | VERSION 1.0
October 2026 · BOT Chain · Chain ID 968

PROTOCOL OVERVIEW

DuesPay is a smart-contract protocol for transparent, trustless dues collection deployed on BOT Chain. It replaces manual payment tracking and treasurer-controlled bank accounts with a fully on-chain campaign system in which every payment, balance change, and expense is permanently and publicly auditable. The protocol requires zero infrastructure beyond a deployed Solidity contract and a static frontend.

A technical overview of the protocol, its design, core functions, security properties, and roadmap.

1. Problem Statement

Student associations, professional bodies, cooperatives, and class groups routinely collect membership dues. Today this is done through manual collection, centralised platforms, or treasurer trust — approaches that suffer from opacity, error, and single-point-of-failure risk. There is no permissionless way to verify that dues were collected in full and spent appropriately.

- Manual collection — cash or bank transfer tracked in spreadsheets. No auditability, fraud risk.
- Centralised platforms — require accounts, charge fees, and can freeze funds.
- Treasurer trust — members must trust a single person to handle all funds correctly.

2. Solution

DuesPay encodes the entire dues lifecycle in a single Solidity contract deployed on BOT Chain:

- Create — Treasurer deploys a named campaign with a fixed per-member amount and a deadline.
- Collect — Members send the exact required amount before the deadline; the contract rejects any other value.
- Spend — Treasurer withdraws to pay for approved expenses. Every withdrawal records a description on-chain.
- Audit — Anyone can query paid status, member lists, expenses, and balances at any time.

3. Contract Architecture

3.1 Data Model

The contract stores two primary structs:

Campaign: title, treasurer address, amount (wei), deadline (unix timestamp), collected balance, spent balance, paused flag.

Expense: campaignId, recipient address, amount, description string, timestamp.

3.2 Key Invariants

`Invariant`
Solidity Enforcement

Exact payment

require(msg.value == c.amountWei)

One payment per member

require(!paid[id][msg.sender])

No overspending

require(c.spent + amount <= c.collected)

Treasurer-only spend

require(msg.sender == c.treasurer)

Deadline enforced

require(block.timestamp <= c.deadline)

3.3 Write Functions

`Function`

Role	Description
create(title, amountWei, deadline)	Any	Opens a new dues campaign
payDues(id)	Member	Pays dues — exact ETH value required
withdraw(id, to, amount, desc)	Treasurer	Withdraws for a named expense
setPaused(id, paused)	Treasurer	Pauses / unpauses new payments

3.4 Read / View Functions

Function	Returns
campaigns(id)	Full campaign struct
paid(id, address)	True if address has paid
campaignCount()	Total number of campaigns
getMembers(id)	Array of addresses that paid
memberCount(id)	Count of paying members

Function	Returns
balance(id)	Available (collected – spent)
expenseCount()	Total expenses across all campaigns
expenses(id)	Expense struct by index

4. Security Properties

4.1 Reentrancy Protection

withdraw() uses the checks-effects-interactions pattern. c.spent is incremented and the Expense record is pushed before the ETH transfer, preventing reentrancy exploits.

4.2 Integer Overflow

The contract targets Solidity >=0.8.20 where arithmetic is checked by default — overflow/underflow reverts automatically.

4.3 Access Control

The treasurer address is immutable after campaign creation. There is no admin role and no upgrade mechanism, minimising attack surface.

4.4 Pause Mechanism

The paused flag allows the treasurer to stop incoming payments temporarily without losing any state — useful while a membership dispute or deadline extension is resolved.

5. Event Log

All state changes emit events, enabling off-chain indexers and explorers to reconstruct the full history:

Event	Parameters
CampaignCreated	id, title, treasurer, amountWei, deadline
DuesPaid	id, member, amount
Withdrawn	id, expenseId, to, amount, description
CampaignPaused	id, paused

6. Deployment & Integration

6.1 Network

Parameter	Value
Chain ID	968
RPC Endpoint	https://rpc.botchain.ai

Parameter	Value
Block Explorer	https://scan.botchain.ai
Compiler	Solidity 0.8.20+

6.2 Deployment Steps

- Compile contract/DuesPay.sol with Solidity 0.8.20+.
- Deploy via Remix, Hardhat, or Foundry on BOT Chain (Chain ID 968).
- Copy the deployed address into js/config.js.

- Serve the static frontend with any web server (e.g. VS Code Live Server).

7. Limitations & Disclaimer

- Not audited. Use on testnet before any mainnet deployment.
- No partial payments — members must send exactly the required amount.
- No refund mechanism — the contract does not return funds if a campaign under-collects.
- Treasurer key security — whoever holds the private key controls all withdrawals. Consider a Gnosis Safe.

8. Roadmap

Milestone	Description
v1.1	Multi-treasurer approval (2-of-3 multisig pattern)
v1.2	ERC-20 token support (USDC, stablecoins)
v1.3	Partial-payment plans (instalments)
v2.0	Factory contract for one-click campaign creation